

컴퓨터 네트워크 과제 [선택 1]

TCP Socket HTTP Server/Client

Python TCP Socket 기반 HTTP Request/Response 및 파일 제어 구현 보고서

환경: macOS, localhost 127.0.0.1, port 8080 | 캡처: Wireshark lo0

목차

1. 과제 개요
2. 구현 목적
3. 개발 환경
4. 사용 기술
5. 전체 프로그램 구조
6. TCP Socket 통신 흐름
7. HTTP Request 메시지 구조
8. HTTP Response 메시지 구조
9. 구현한 Method-Response Case
10. server.py 코드 설명
11. client.py 코드 설명
12. 실행 방법
13. GET 실행 결과 설명
14. POST 실행 결과 설명
15. PUT 실행 결과 설명
16. DELETE 및 오류 처리 결과 설명
17. Client / WireShark 실행 결과
18. Server 실행 결과
19. 제출 파일 및 캡처 파일 구성
20. 전체 실행 결과 요약

1. 과제 개요

본 과제는 Python으로 TCP 기반 소켓 Server와 Client 프로그램을 작성하고, Client가 HTTP 형식의 Request를 직접 생성하여 Server로 전송하는 프로젝트이다.

Server는 Python socket 모듈만 사용하여 TCP 연결을 수락하고, Request Line을 파싱하여 Method, Path, Version을 분리한다. 이후 GET, HEAD, POST, PUT, DELETE, 잘못된 Method 요청에 따라 HTTP Response를 직접 구성한다.

실행 환경은 macOS, localhost 127.0.0.1, port 8080을 기준으로 하며, Wireshark에서는 loopback 인터페이스인 lo0에서 tcp.port == 8080 필터로 패킷을 캡처한다.

2. 구현 목적

웹 프레임워크 없이 TCP Socket만으로 HTTP 메시지 흐름을 구현하여 응용 계층 프로토콜이 실제 네트워크 위에서 어떻게 동작하는지 이해하는 것이 목적이다.

GET은 파일 조회, POST는 파일 생성 및 append, PUT은 파일 덮어쓰기, DELETE는 파일 삭제 기능과 연결하였다. 따라서 단순한 문자열 응답이 아니라 실제 파일 제어를 수행하는 Server 중심 구현이다.

HEAD는 과제 기본 Method 확인용으로 포함하되, 영상 시연과 설명에서는 교수님 코멘트에 맞춰 GET, POST, PUT, DELETE 및 파일 제어 흐름을 중심으로 설명한다.

3. 개발 환경

운영체제: macOS

언어: Python 3

통신 방식: TCP Socket

주소 및 포트: 127.0.0.1:8080

캡처 도구: Wireshark

제출 파일: server.py, client.py, index.html, README.md, README.docx, README.pdf

4. 사용 기술

socket.AF_INET은 IPv4 주소 체계를 의미하고 socket.SOCK_STREAM은 TCP 스트림 소켓을 의미한다.

Server는 bind(), listen(), accept(), recv(), sendall()을 직접 사용한다. Client도 TCP 연결 후 sendall()과 recv()로 HTTP 형식 메시지를 주고받는다.

Response는 HTTP/1.1 status line, Date, Server, Content-Type, Content-Length, Connection header, blank line, body 순서로 직접 작성한다.

5. 전체 프로그램 구조

server.py: TCP Server, Request 파싱, Method 분기, 파일 제어, Response 생성 담당

client.py: 사용자 입력 기반 HTTP Request 생성, TCP 전송, Response 출력 담당

index.html: GET /index.html 요청에 대한 200 OK 테스트 파일

README.md: GitHub 및 제출용 설명 문서

README.docx / README.pdf: 과제 제출용 10장 이상 상세 보고서

README/: 본인 PC에서 직접 캡처한 실행 결과 PNG를 저장할 폴더

WireShark/: 본인 PC에서 직접 저장한 Wireshark pcapng 파일을 저장할 폴더

6. TCP Socket 통신 흐름

Server는 socket()으로 TCP 소켓을 만들고 bind()로 127.0.0.1:8080에 연결한다.

listen()으로 연결 요청을 기다린 후 accept()로 Client 연결을 수락한다.

Client는 socket()으로 TCP 소켓을 만들고 connect()로 Server에 접속한다.

Client는 sendall()로 HTTP Request를 전송하고, Server는 recv()로 Request를 수신한다.

Server는 처리 결과를 sendall()로 전송하고, Client는 recv()를 반복하여 Response 전체를 출력한다.

7. HTTP Request 메시지 구조

Request Line 예시: GET /index.html HTTP/ 1.1

Header에는 Host, User-Agent, Content-Type, Content-Length, Connection을 포함한다.

Header와 Body 사이에는 반드시 blank line이 있어야 하며, 이 프로젝트에서는 CRLF 문자 조합을 사용하여 HTTP 형식에 맞춘다.

POST와 PUT은 Body를 입력받아 Content-Length를 byte 길이로 계산한 뒤 전송한다.

8. HTTP Response 메시지 구조

Response Status Line 예시: HTTP/ 1.1 200 OK

Server는 Date, Server, Content-Type, Content-Length, Connection header를 직접 구성한다.

GET, POST, DELETE 오류 응답은 Body를 포함할 수 있고, HEAD와 204 No Content 응답은 Body 없이 header 중심으로 응답한다.

Wireshark에서는 이 구조가 HTTP format으로 해석되어 Request Line과 Status Line을 확인할 수 있다.

9. 구현한 Method-Response Case

No	Request	Response	기능
1	GET /index.html	200 OK	HTML 파일 조회
2	GET /notfound.html	404 Not Found	없는 파일 오류
3	POST /post.txt	201 Created	파일 생성
4	POST /post.txt	200 OK	파일 append
5	PUT /put.txt	204 No Content	파일 덮어쓰기
6	PUT /readonly.txt	403 Forbidden	수정 금지
7	DELETE /post.txt	200 OK	파일 삭제
8	DELETE /readonly.txt	403 Forbidden	삭제 금지
9	PATCH /index.html	400 Bad Request	잘못된 Method

GET /index.html -> 200 OK: index.html 파일 조회 성공

GET /notfound.html -> 404 Not Found: 없는 파일 요청

HEAD /index.html -> 200 OK: Body 없이 header만 응답

HEAD /missing.html -> 404 Not Found: 없는 파일에 대한 HEAD 응답

POST /post.txt -> 201 Created: 파일이 없으면 새로 생성

POST /post.txt -> 200 OK: 파일이 있으면 기존 내용 뒤 append

PUT /put.txt -> 204 No Content: 파일 내용 덮어쓰기

PUT /readonly.txt -> 403 Forbidden: 수정 금지 처리

DELETE /post.txt -> 200 OK: 파일 삭제 성공

DELETE /readonly.txt -> 403 Forbidden: 삭제 금지 처리

PATCH /index.html -> 400 Bad Request: 지원하지 않는 Method 처리

10. server.py 코드 설명

`parse_http_request()`는 TCP로 받은 bytes를 UTF-8 문자열로 변환하고 request line, headers, body로 분리한다.

`receive_full_request()`는 TCP가 스트림 방식이라는 점을 고려하여 CRLF CRLF가 나올 때까지 header를 수신하고, Content-Length만큼 body를 추가 수신한다.

`safe_file_path()`는 URL 경로를 프로젝트 폴더 내부 파일 경로로 변환하고, 상위 경로 이동을 막아 안전하게 파일을 제어한다.

`handle_get()`, `handle_post()`, `handle_put()`, `handle_delete()`는 Method별 실제 동작을 수행한다.

`build_http_response()`는 HTTP status line과 header, blank line, body 구조를 직접 만들어 bytes로 반환한다.

`run_server()`는 bind, listen, accept, recv, sendall을 사용하고 KeyboardInterrupt로 안전하게 종료된다.

`make_http_date()`: Response Header의 Date 값을 GMT 형식으로 생성한다.

`guess_content_type()`: .html 요청은 text/html, .txt 요청은 text/plain으로 Content-Type을 지정한다.

`handle_get()`: 파일이 존재하면 200 OK와 파일 내용을 응답하고, 없으면 404 Not Found를 응답한다.

`handle_head()`: GET과 같은 상태 판단을 하되 Body 없이 Header만 응답한다.

`handle_post()`: /post.txt가 없으면 생성 후 201 Created, 있으면 append 후 200 OK를 응답한다.

`handle_put()`: /put.txt를 Body 내용으로 덮어쓰고 204 No Content를 응답한다. /readonly.txt는 403 Forbidden으로 거부한다.

`handle_delete()`: /post.txt 또는 /put.txt를 삭제하고 200 OK를 응답한다. 삭제 금지 대상은 403 Forbidden으로 처리한다.

Server 터미널에는 Client 주소, 원본 Request, Parsed method/path/version, Header count, Body length, Response status가 출력된다.

영상 설명에서는 Server가 단순 문자열 출력 프로그램이 아니라 Request 파싱, 파일 제어, HTTP Response 생성을 수행한다는 점을 강조한다.

11. client.py 코드 설명

client.py는 Method와 Path를 사용자에게 입력받는다. POST와 PUT이면 Body도 추가 입력받는다.

build_request()는 Request Line, Host, User-Agent, Content-Type, Content-Length, Connection header를 직접 만든다.

send_request()는 TCP로 Server에 연결하여 Request를 전송하고 Response 전체를 수신한다.

Client 터미널에는 보낸 Request와 받은 Response가 모두 출력되므로 영상 시연에서 HTTP 메시지 확인이 쉽다.

사용자가 입력한 Method와 Path는 Request Line으로 변환된다. 예: GET /index.html HTTP/1.1

POST와 PUT은 Body 입력을 추가로 받고, Body의 UTF-8 byte 길이를 계산하여 Content-Length에 넣는다.

Client는 socket.AF_INET, socket.SOCK_STREAM으로 TCP 소켓을 만들고 127.0.0.1:8080에 connect한다.

sendall()로 Request 전체를 전송하고, recv()를 반복하여 Server가 보낸 Response를 끝까지 수신한다.

영상 설명에서는 Client 출력의 Sent Request와 Received Response가 Wireshark 패킷의 HTTP 내용과 일치한다는 점을 보여준다.

12. 실행 방법

터미널 1에서 프로젝트 폴더로 이동한 뒤 python3 server.py를 실행한다.

터미널 2에서 같은 폴더로 이동한 뒤 python3 client.py를 실행한다.

Server가 먼저 실행되어 있어야 Client가 127.0.0.1:8080으로 접속할 수 있다.

13. GET 실행 결과 설명

GET /index.html은 index.html 파일을 읽어 200 OK와 HTML Body를 반환한다.

GET /notfound.html은 파일이 없으므로 404 Not Found를 반환한다.

POST 또는 PUT 후 GET /post.txt, GET /put.txt를 실행하면 파일 제어 결과가 실제로 반영되었는지 확인할 수 있다.

14. POST 실행 결과 설명

첫 번째 POST /post.txt는 post.txt가 없으면 파일을 생성하고 201 Created를 반환한다.

두 번째 POST /post.txt는 파일이 이미 존재하므로 기존 내용 뒤에 새로운 Body를 append하고 200 OK를 반환한다.

이후 GET /post.txt를 실행하면 두 줄의 내용이 모두 출력되어 append 결과를 확인할 수 있다.

15. PUT 실행 결과 설명

PUT /put.txt는 Body 내용으로 put.txt를 덮어쓴다.

응답은 204 No Content이다. 이것은 실패가 아니라 요청은 성공했지만 응답 Body가 없다는 뜻이다.

영상에서는 PUT 직후 Server 로그의 Response status와 GET /put.txt 결과를 함께 보여주면 파일 덮어쓰기가 성공했음을 명확히 설명할 수 있다.

16. DELETE 및 오류 처리 결과 설명

DELETE /post.txt는 파일이 존재하면 삭제하고 200 OK를 반환한다.

DELETE /readonly.txt는 삭제 금지 대상으로 가정하여 403 Forbidden을 반환한다.

PATCH /index.html은 지원하지 않는 Method이므로 400 Bad Request를 반환한다.

17. Client / Wireshark 실행 결과

GET /index.html 실행 결과, Client는 HTTP/1.1 형식의 GET Request를 전송하고 Server는 index.html 파일 내용을 포함하여 200 OK로 응답한다. README/GET_OK.png에는 Client 결과 또는 Wireshark HTTP 패킷 화면을 첨부한다.

GET /notfound.html 실행 결과, 존재하지 않는 파일 요청에 대해 Server는 404 Not Found를 응답한다. README/GET_NO.png에는 404 응답 화면을 첨부한다.

HEAD /index.html 실행 결과, Server는 200 OK Header를 보내지만 Body를 전송하지 않는다. README/HEAD.png에는 Header 중심 응답 화면을 첨부한다.

POST /post.txt 첫 번째 실행 결과, Server는 post.txt 파일을 생성하고 201 Created를 응답한다. README/POST_1.png에는 Request Body와 201 응답 화면을 첨부한다.

POST /post.txt 두 번째 실행 결과, Server는 기존 post.txt 뒤에 Body를 append하고 200 OK를 응답한다. README/POST_2.png에는 append 응답 화면을 첨부한다.

PUT /put.txt 실행 결과, Server는 Body 내용으로 put.txt를 덮어쓰고 204 No Content를 응답한다. README/PUT_OK.png에는 204 응답과, 필요 시 GET /put.txt 확인 결과를 함께 첨부한다.

DELETE /post.txt 실행 결과, Server는 생성된 파일을 삭제하고 200 OK를 응답한다. README/DELETE.png에는 DELETE 요청과 200 OK 응답 화면을 첨부한다.

PATCH /index.html 실행 결과, 지원하지 않는 Method이므로 Server는 400 Bad Request를 응답한다. README/BAD_REQUEST.png에는 잘못된 Method 처리 결과를 첨부한다.

Wireshark 원본 캡처 파일은 WireShark/GET_OK.pcapng, WireShark/POST_1.pcapng, WireShark/PUT.pcapng, WireShark/DELETE.pcapng처럼 Method별로 저장한다.

18. Server 실행 결과

Server 실행 시 터미널에는 Listening 메시지가 출력되어 127.0.0.1:8080에서 Client 연결을 기다리는 상태임을 확인할 수 있다.

Client가 접속하면 Server는 Client 주소를 출력하고, 원본 Request 메시지를 그대로 출력한다.

Server는 Request Line을 분석하여 Parsed method, Parsed path, Parsed version을 출력한다.

POST와 PUT 요청에서는 Body length가 0보다 크게 출력되어 본문 데이터가 Server에 정상 전달되었음을 확인할 수 있다.

각 요청 처리 후 Server는 Response status를 출력한다. 예를 들어 GET 성공은 HTTP/1.1 200 OK, POST 생성은 HTTP/1.1 201 Created, PUT 성공은 HTTP/1.1 204 No Content, 잘못된 Method는 HTTP/1.1 400 Bad Request로 출력된다.

Server 결과 화면은 README/SERVER_RESULT.png로 저장하여 Client/Wireshark 결과와 함께 제출 자료에 포함할 수 있다.

19. 제출 파일 및 캡처 파일 구성

소스 파일은 server.py와 client.py이다. index.html은 GET 및 HEAD 성공 응답을 검증하기 위한 테스트 파일이다.

상세 보고서는 README.docx와 README.pdf로 제공한다. 두 파일은 GitHub 기본 README보다 자세한 코드 설명, 환경, 실행 결과 설명을 포함한다.

README 폴더에는 실행 결과 PNG 이미지를 저장한다. 예: GET_OK.png, GET_NO.png, HEAD.png, POST_1.png, POST_2.png, PUT_OK.png, DELETE.png, BAD_REQUEST.png, SERVER_RESULT.png

WireShark 폴더에는 Wireshark에서 저장한 원본 pcapng 파일을 저장한다. 예: GET_OK.pcapng, GET_NO.pcapng, HEAD.pcapng, POST_1.pcapng, POST_2.pcapng, PUT.pcapng, DELETE.pcapng, BAD_REQUEST.pcapng

실행 중 생성되는 post.txt, put.txt, __pycache__ 등은 결과 파일이므로 GitHub에는 포함하지 않도록 .gitignore에서 제외한다.

20. 전체 실행 결과 요약

본 프로젝트는 Python TCP Socket을 직접 사용하여 HTTP 형식의 Server/Client 프로그램을 구현하였다.

Server는 GET, HEAD, POST, PUT, DELETE, 잘못된 Method를 처리하며 파일 조회, 생성, append, 덮어쓰기, 삭제 기능을 수행한다.

Client는 사용자 입력으로 HTTP Request를 생성하고, 전송한 Request와 수신한 Response 전체를 출력한다.

Wireshark 캡처를 통해 TCP 연결, HTTP Request, HTTP Response가 실제 패킷으로 교환되는 것을 확인할 수 있다.

총 11개의 Method-Response case를 구현하여 과제에서 요구한 5개 이상의 case 조건을 충족한다.